DataPyme

Implementación del LiDAR Unitree L2

Del sensor nuevo al mapa de una habitación

Guía de implementación para el robot humanoide Unitree R1 EDU

Versión 0.1.0

Proyecto	Programación del robot humanoide Unitree R1 EDU
Colaboración	UTN Facultad Regional San Francisco
Equipo	Unitree 4D LiDAR L2 · PC con Ubuntu 24.04 + ROS 2 Jazzy
Repositorio	github.com/EzeCombina/LiDAR-RobotHumanoide (privado)
Autor	Ezequiel Combina
Fecha	Noviembre de 2026

Proyecto realizado en colaboración con la **UTN Facultad Regional San Francisco**

UTN UNIVERSIDAD TECNOLOGICA NACIONAL | FACULTAD REGIONAL SAN FRANCISCO

Contenido

1. Resumen **4**
1.1 Qué se logra 4
1.2 Para quién es 4
1.3 Qué hace falta 5
1.4 Alcance de esta versión 5

2. Conceptos básicos **6**
2.1 LiDAR 6
2.2 Nube de puntos 6
2.3 IMU 6
2.4 ROS 2: nodos, tópicos y mensajes 6
2.5 Marcos de referencia 7
2.6 Grabaciones (rosbag) 7
2.7 SLAM 7
2.8 Mapas 3D y 2D 7

3. Materiales y requisitos **8**
3.1 El sensor: Unitree 4D LiDAR L2 8
3.2 La computadora 8
3.3 Software 9
3.4 Software de Windows (Unilidar 2) 9

4. Instalación del entorno **10**
4.1 Herramientas base 10
4.2 ROS 2 Jazzy 10
4.3 Librerías del sensor, del SLAM y de los mapas 11
4.4 Repositorio del proyecto 12
4.5 Verificación: relevamiento del sistema 12

5. Conexión y puesta en marcha del sensor **13**
5.1 Conexión física 13
5.2 Configuración de red 13
5.3 Verificar la comunicación 14
5.4 SDK del sensor y programa de ejemplo 14
5.5 Estado interno del sensor 15
5.6 Vibración del sensor 16
5.7 Conexión por USB (pendiente de probar) 16

6. Driver de ROS 2 y visualización **17**
6.1 Compilar el driver 17
6.2 Lanzar el driver 17
6.3 Verificar los tópicos 18
6.4 Ver la nube en RViz 18

7. Grabación de datos **20**
7.1 Grabar 20

7.2 Revisar una grabación . . . 21
7.3 Ver una grabación en RViz . . . 21

8. SLAM y generación del mapa 3D . . . 23
8.1 Descargar y compilar Point-LIO . . . 23
8.2 Configuración para el L2 . . . 23
8.3 Generar el mapa de una grabación . . . 24
8.4 Lo mismo, a mano . . . 24
8.5 Ver y medir el mapa . . . 25

9. Mapeo de una habitación . . . 26
9.1 Preparación . . . 26
9.2 Cómo llevar el sensor . . . 26
9.3 Grabar el recorrido . . . 26
9.4 Generar, revisar y ver el mapa . . . 27
9.5 Resultado obtenido . . . 27

10. Procesamiento del mapa: mapa 3D limpio y mapa 2D . . . 28
10.1 Mapa 3D limpio . . . 28
10.2 Mapa 2D de ocupación para Nav2 . . . 29
10.3 Verificar el mapa 2D con Nav2 . . . 30

11. Problemas frecuentes . . . 32
11.1 Instalación y repositorio . . . 32
11.2 Red y sensor . . . 32
11.3 Driver y RViz . . . 33
11.4 Grabación . . . 33
11.5 SLAM y mapas . . . 33

Anexo A. Parámetros . . . 35
A.1 Launch del driver (config/unitree_lidar_ros2/l2_driver.launch.py) . . . 35
A.2 Point-LIO (config/unilidar_l2.yaml y mapping_unilidar_l2.launch.py) . . . 35
A.3 Scripts del proyecto . . . 35

Anexo B. Versiones . . . 37
B.1 Sensor y repositorios . . . 37
B.2 Sistema . . . 37
B.3 Paquetes apt . . . 37

Anexo C. Archivos del repositorio . . . 38
C.1 Regenerar este documento . . . 38

Anexo D. Referencias . . . 40
D.1 Documentación del fabricante . . . 40
D.2 Software . . . 40
D.3 Publicaciones . . . 40

1. Resumen

Esta guía explica, paso por paso, cómo poner en marcha el **LiDAR Unitree 4D L2** en una computadora con **Ubuntu 24.04 y ROS 2 Jazzy**, desde el sensor recién sacado de la caja hasta obtener el **mapa de una habitación**: un mapa 3D (nube de puntos) y un mapa 2D listo para el sistema de navegación del robot.

Es parte del proyecto de **DataPyme** de programación del robot humanoide **Unitree R1 EDU**, que se desarrolla en conjunto con la **UTN Facultad Regional San Francisco**. El LiDAR se va a montar sobre el robot para que pueda mapear el lugar donde trabaja y, más adelante, navegar solo. Este documento cubre la primera parte: aprender a usar el sensor y a generar mapas, llevándolo en la mano.

1.1 Qué se logra

- **Sensor funcionando en Ubuntu:** red configurada, datos verificados y diagnóstico del estado interno del sensor (motores, temperaturas, tensiones).
- **Driver de ROS 2** del sensor compilado y funcionando en Jazzy, con la nube de puntos en RViz.
- **Grabación** de los datos del sensor con controles automáticos (cortes de datos, arranque quieto).
- **Mapa 3D** del lugar con el SLAM Point-LIO, a partir de una grabación.
- **Productos para el robot:** mapa 3D limpio y nivelado, y **mapa 2D de ocupación** en el formato de Nav2.

Resultado obtenido con una habitación recorrida con el sensor en la mano: mapa de unos 3,2 × 7,9 m, con un **error de cierre de 10,5 cm** (diferencia entre el punto de llegada y el de partida, que eran el mismo) y piso nivelado a menos de 1°.

1.2 Para quién es

Para estudiantes y docentes del laboratorio con conocimientos **básicos, medios o avanzados**. No hace falta saber ROS 2 de antemano: los conceptos nuevos se explican en recuadros como este:

> **Recuadro de concepto**
> Explicación breve de una idea nueva. Quien ya la conoce puede saltearla sin perder el hilo.

Los otros recuadros que aparecen en la guía:

> **Nota**
> Datos útiles, aclaraciones o dónde encontrar algo.

> **Importante**
> Algo que, si se pasa por alto, hace que el resultado salga mal.

> **Peligro**
> Riesgo de dañar el sensor, la computadora o el robot.

1.3 Qué hace falta

- El **LiDAR Unitree L2** con su fuente de 12 V, el módulo adaptador y un cable Ethernet.
- Una **PC con Ubuntu 24.04** con un puerto Ethernet libre (capítulo 3).
- Conexión a internet para instalar el software, y acceso al repositorio del proyecto.
- Unas horas para la instalación y las primeras pruebas.

Todo el software propio del proyecto (scripts, configuraciones y documentación) está en el repositorio `github.com/EzeCombina/LiDAR-RobotHumanoide`. Cada capítulo indica qué script usar y, además, **cómo hacer lo mismo a mano**, para entender qué hace cada herramienta.

1.4 Alcance de esta versión

Tema	Estado
Implementación en una PC con Ubuntu, conexión por **Ethernet**	Completo y verificado
Mapeo de una habitación con el sensor en la mano	Completo y verificado
Mapa 3D limpio y mapa 2D para Nav2	Completo y verificado
Conexión por **USB** (modo serie del sensor)	Pendiente: herramientas listas, sin probar
Implementación en la **Jetson** del robot R1	Próximo documento
Navegación autónoma, mapa en tiempo real, simulación en Gazebo	Próximo documento

> **Dónde encontrar más detalle**
>
> La documentación de trabajo del proyecto está en el repositorio: `docs/pasos/` (un documento por paso), `docs/bitacora.md` (registro de cada sesión, con los errores encontrados y cómo se resolvieron) y `docs/versiones.md` (versiones exactas de todo lo usado).

2. Conceptos básicos

Este capítulo reúne las ideas que se usan en toda la guía. Quien ya trabajó con LiDAR y ROS 2 puede pasar directamente al capítulo 3.

2.1 LiDAR

Un **LiDAR** (*Light Detection And Ranging*) mide distancias con pulsos de luz láser: emite un pulso, lo recibe reflejado y calcula la distancia por el tiempo que tardó en volver. Para medir en muchas direcciones, mueve el haz.

El **L2** tiene dos motores:

- uno de **alta velocidad** que hace girar un espejo y barre en vertical (216 veces por segundo);
- uno de **baja velocidad** que hace girar todo el cabezal y barre en horizontal.

Así cubre una **semiesfera de 360° × 90°** por encima de su base y mide unos **64.000 puntos por segundo**, de 5 cm a 30 m de distancia.

> **El L2 solo ve hacia arriba de su base**
>
> Apoyado horizontal, con la cúpula hacia arriba, ve paredes y techo pero **no ve el piso** ni nada que esté más abajo que él. Esto influye en cómo conviene llevarlo para mapear (capítulo 9).

2.2 Nube de puntos

Cada medición del LiDAR es un **punto** con coordenadas X, Y, Z (en metros) y una intensidad (cuánta luz volvió). El conjunto de puntos de un barrido se llama **nube de puntos**. El L2 entrega unas 12 nubes por segundo, de unos 4.000 a 5.000 puntos cada una.

Las coordenadas de cada nube están expresadas **respecto del sensor en ese instante**: el punto (0, 0, 0) es el centro de la base del sensor.

2.3 IMU

El L2 trae una **IMU** (*Inertial Measurement Unit*, unidad de medición inercial): un acelerómetro y un giróscopo que miden cómo se mueve el sensor, unas 250 veces por segundo. Con el sensor quieto, el acelerómetro mide la **gravedad** (unos 9,8 m/s²), y eso indica hacia dónde está "abajo".

2.4 ROS 2: nodos, tópicos y mensajes

ROS 2 (*Robot Operating System 2*) no es un sistema operativo: es un conjunto de librerías y herramientas para que distintos programas de un robot intercambien datos.

- Un **nodo** es un programa (por ejemplo, el driver del LiDAR).
- Un **tópico** es un canal con nombre por el que un nodo **publica** mensajes y otros nodos los **reciben** (se **suscriben**). El driver del L2 publica la nube en `/unilidar/cloud` y la IMU en `/unilidar/imu`.
- Un **mensaje** tiene un tipo fijo: la nube es `sensor_msgs/PointCloud2` y la IMU, `sensor_msgs/Imu`.

- Cada versión de ROS 2 se llama **distribución** (Foxy, Humble, **Jazzy**...) y corresponde a una versión de Ubuntu. Esta guía usa **Jazzy** con **Ubuntu 24.04**.

RViz es el visualizador 3D de ROS 2: se suscribe a los tópicos y los dibuja.

2.5 Marcos de referencia

Cada dato indica en qué sistema de coordenadas (**marco**, *frame*) está expresado. La nube del L2 está en el marco `unilidar_lidar` y los mapas, en el marco `map`. ROS 2 publica las relaciones entre marcos en el tópico `/tf`.

2.6 Grabaciones (rosbag)

Un **rosbag** es un archivo donde ROS 2 guarda los mensajes de uno o varios tópicos, con su hora exacta. Después se puede **reproducir**: ROS 2 vuelve a publicar los mensajes como si el sensor estuviera conectado. En Jazzy el formato es **MCAP** (`.mcap`). Grabar permite generar el mapa después, en la PC, y repetir el procesamiento con otros parámetros sin volver a recorrer el lugar.

2.7 SLAM

Si el sensor se mueve, las nubes no se pueden sumar directamente: cada una está expresada respecto del sensor en su propio instante. Un **SLAM** (*Simultaneous Localization And Mapping*, localización y mapeo simultáneos) calcula **dónde está el sensor** en cada momento (su **trayectoria**) y, con eso, ubica todas las nubes en un mismo sistema de coordenadas y las suma en un **único mapa**.

Esta guía usa **Point-LIO**, un SLAM que combina la nube del LiDAR con la IMU (*LiDAR-Inertial Odometry*).

2.8 Mapas 3D y 2D

- **Mapa 3D:** la nube de puntos de todo el lugar, en un archivo `.pcd` (*Point Cloud Data*). Sirve para ver el lugar, medirlo y, en el futuro, para ubicar al robot en 3D.
- **Mapa 2D de ocupación:** una grilla vista desde arriba, como una hoja cuadriculada, donde cada celda está **ocupada** (negro), **libre** (blanco) o **desconocida** (gris). Se guarda como una imagen (`.pgm`) más un archivo con la escala (`.yaml`). Es el formato que usa **Nav2**, el sistema de navegación de ROS 2, para que el robot se ubique y planifique sus recorridos.

3. Materiales y requisitos

3.1 El sensor: Unitree 4D LiDAR L2

Característica	Valor
Campo de visión	360° × 90° (semiesfera por encima de la base); 360° × 96° en modo de ángulo negativo
Puntos por segundo	64.000 (unas 12 nubes por segundo)
Distancia de medición	0,05 a 30 m (con 90 % de reflectividad)
IMU	Integrada, unas 250 mediciones por segundo, con los ejes paralelos a los de la nube
Alimentación	**12 V DC**, 1 A; 10 W típicos, picos de 13 W mientras se calienta
Interfaces de datos	Ethernet (UDP, modo de fábrica) o serie UART por USB (un solo modo activo a la vez)
Fijación	4 tornillos M3 en la base
Peso y medidas	230 g; 75 × 75 × 65 mm
Sistema de coordenadas	Origen en el centro de la cara inferior; eje +X opuesto a la salida del cable

Fuentes: manual oficial del L2 (v1.1) y manual de Unilidar 2 (v1.0), guardados en `docs/referencias/`.

Contenido de la caja

- El sensor, con su cable de salida.
- Un **módulo adaptador** con tres conectores: entrada de alimentación (plug DC de 3,5 × 1,35 mm), **RJ45** (Ethernet) y **USB-C** (modo serie).
- Una **fuente de 12 V / 1 A**.
- Una **almohadilla de goma** para apoyar el sensor mientras se lo prueba.

> **Alimentación: solo 12 V**
>
> El L2 se alimenta con **12 V**. No conectarlo nunca a 24 V ni a 36 V (por ejemplo, a las salidas XT30 del robot R1): para usarlo en el robot hace falta un conversor DC-DC a 12 V. El manual no especifica la polaridad del plug: si se usa una fuente distinta de la original, **medirla antes** (capítulo 5).

Versiones del sensor del laboratorio

Dato	Valor
Hardware	2.2.1.1
Firmware	2.8.11.1
Red de fábrica	Sensor `192.168.1.62` → envía los datos a `192.168.1.2`, puertos UDP 6101 → 6201

3.2 La computadora

Cualquier PC con **Ubuntu 24.04 LTS** y arquitectura x86_64 sirve. Lo importante:

- **Un puerto Ethernet libre**, que queda dedicado al sensor. Internet se usa por Wi-Fi o por otra placa.

- **Espacio en disco:** unos 10 GB para el software, más unos **120 MB por minuto** de grabación.
- **Placa de video:** cualquiera que maneje RViz (el visualizador 3D).

PC usada para esta guía:

Elemento	Valor
Procesador / memoria	AMD Ryzen 5 4600H (12 hilos) / 22 GiB
Sistema	Ubuntu 24.04.5 LTS, kernel 7.0.0-42
ROS 2	Jazzy (`ros-jazzy-desktop` 0.11.0)
Compilador	gcc 13.3.0, CMake 3.28.3, Python 3.12.3

3.3 Software

Componente	Origen	Uso
ROS 2 Jazzy	Paquetes oficiales (apt)	Base de todo: comunicación, RViz, grabación
SDK del L2 (`unilidar_sdk2` v2.0.10)	`github.com/unitreerobotics/unilidar_sdk2`	Librería del sensor, ejemplos y **driver ROS 2**
Point-LIO (`point_lio_ros2`)	`github.com/dfloreaa/point_lio_ros2`	SLAM: genera el mapa 3D
Nav2 map server	Paquetes oficiales (apt)	Cargar y verificar el mapa 2D
Scripts del proyecto	Repositorio `EzeCombina/LiDAR-RobotHumanoide`	Red, diagnóstico, grabación, mapas

Las versiones exactas (paquetes y *commits* de cada repositorio) están en el **Anexo B**.

> **Jazzy y el driver de Unitree**
>
> Unitree verificó su driver de ROS 2 solo en Ubuntu 20.04 con ROS 2 Foxy, y el autor de Point-LIO para ROS 2, en Ubuntu 22.04 con Humble. En este proyecto se comprobó que **ambos compilan y funcionan sin cambios en Ubuntu 24.04 con Jazzy**: no hace falta usar contenedores ni otra versión de ROS 2.

3.4 Software de Windows (Unilidar 2)

Unitree ofrecía un programa para Windows, **Unilidar 2**, para ver la nube y configurar el sensor. A noviembre de 2026 **ya no se puede descargar** de su página. Todo lo que hacía (ver la nube, consultar el estado, cambiar de modo) se hace en esta guía con el SDK y herramientas propias en Ubuntu.

4. Instalación del entorno

Este capítulo parte de una PC con **Ubuntu 24.04 recién instalada** y la deja lista para trabajar con el sensor. Todo se hace desde una terminal (**Ctrl+Alt+T**).

> **Comandos con sudo**
>
> Las líneas que empiezan con `sudo` modifican el sistema y piden la contraseña del usuario. Antes de correrlas, leer qué hacen.

4.1 Herramientas base

```
sudo apt update
sudo apt install -y build-essential cmake git curl tcpdump tmux software-properties-common
```

Paquete	Para qué se usa
`build-essential`	Compiladores de C/C++ (`gcc`, `g++`, `make`)
`cmake`	Sistema de compilación del SDK y de los paquetes de ROS 2
`git`	Descargar repositorios
`curl`	Descargar archivos desde la terminal
`tcpdump`	Ver si llegan los paquetes de red del sensor
`tmux`	Mantener vivos los procesos largos (driver, grabaciones) aunque se cierre la terminal o se corte una sesión SSH
`software-properties-common`	Provee `add-apt-repository`, que se usa al instalar ROS 2

4.2 ROS 2 Jazzy

Procedimiento oficial de ROS 2 para Ubuntu 24.04.

1. Idioma del sistema en UTF-8. Verificar con `locale` que `LANG` termine en `.UTF-8` (por ejemplo `es_AR.UTF-8`). Si no:

```
sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

2. Repositorio de paquetes de ROS 2. Se habilita *universe* y se instala `ros2-apt-source`, que agrega a `apt` el repositorio de ROS 2 con su llave de firma:

```
sudo add-apt-repository universe
export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-infrastructure/ros-apt-source/
releases/latest | grep -F "tag_name" | awk -F\" '{print $4}')
echo "$ROS_APT_SOURCE_VERSION"      # debe mostrar un número de versión
curl -L -o /tmp/ros2-apt-source.deb "https://github.com/ros-infrastructure/ros-apt-source/releases/
download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${ROS_APT_SOURCE_VERSION}.$(. /etc/os-release && echo $
{UBUNTU_CODENAME:-${VERSION_CODENAME}})_all.deb"
sudo dpkg -i /tmp/ros2-apt-source.deb
```

3. ROS 2 y herramientas de desarrollo:

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y ros-jazzy-desktop ros-dev-tools
```

`ros-jazzy-desktop` trae ROS 2 completo con RViz; `ros-dev-tools` trae `colcon` (compila paquetes de ROS 2), `rosdep` (instala dependencias) y `vcstool`. El `apt upgrade` es necesario: en una Ubuntu sin actualizar, la instalación puede borrar paquetes del sistema.

4. Cargar ROS 2 en cada terminal. Cada terminal nueva tiene que "cargar" ROS 2 para que existan los comandos `ros2`, `colcon`, etc. Se agrega una vez al final de `~/.bashrc`:

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

5. Verificar. En una terminal `ros2 run demo_nodes_cpp talker` y en otra `ros2 run demo_nodes_cpp listener`. El primero muestra `Publishing: 'Hello World: 1'`... y el segundo `I heard: [Hello World: 1]`... Cortar ambos con **Ctrl+C**.

4.3 Librerías del sensor, del SLAM y de los mapas

```
sudo apt install -y ros-jazzy-pcl-ros ros-jazzy-pcl-conversions libpcl-dev libeigen3-dev
sudo apt install -y python3-numpy python3-scipy python3-pil \
    ros-jazzy-nav2-map-server ros-jazzy-nav2-util
```

Paquete	Para qué se usa
`libpcl-dev`, **`ros-jazzy-pcl-ros`**, **`ros-jazzy-pcl-conversions`**	**PCL** (*Point Cloud Library*) y su conexión con ROS 2: los usan el driver y Point-LIO
`libeigen3-dev`	Álgebra lineal (matrices, rotaciones)
`python3-numpy`, **`python3-scipy`**, **`python3-pil`**	Scripts de Python del proyecto (análisis y productos del mapa)
`ros-jazzy-nav2-map-server`, **`ros-jazzy-nav2-util`**	Cargar y verificar el mapa 2D como lo hace Nav2

Inicializar `rosdep`, una vez por PC (`rosdep update` va **sin** `sudo`):

```
sudo rosdep init
rosdep update
```

4.4 Repositorio del proyecto

Configurar la identidad de git y autenticarse en GitHub (GitHub no acepta la contraseña de la cuenta desde la terminal; la forma más simple es la herramienta `gh`):

```
git config --global user.name  "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"
sudo apt install -y gh
gh auth login        # GitHub.com → HTTPS → Login with a web browser
gh auth setup-git
```

Clonar el repositorio:

```
mkdir -p ~/Documentos && cd ~/Documentos
git clone https://github.com/EzeCombina/LiDAR-RobotHumanoide.git
cd LiDAR-RobotHumanoide
```

> **Carpeta de trabajo**
>
> Todos los comandos de los capítulos siguientes se ejecutan desde `~/Documentos/LiDAR-RobotHumanoide`, salvo que se indique otra carpeta. Los repositorios de terceros (SDK y Point-LIO) se descargan aparte, en `~/lidar_ws/`.

4.5 Verificación: relevamiento del sistema

El script `scripts/sistema/relevar_sistema.sh` muestra todo lo instalado. No modifica nada y no necesita `sudo`:

```
./scripts/sistema/relevar_sistema.sh
```

```
== Sistema operativo ==
Ubuntu 24.04.5 LTS
Kernel: 7.0.0-42-generic  |  Arquitectura: x86_64
...
== ROS 2 ==
Distribuciones en /opt/ros: jazzy
ROS_DISTRO en esta terminal: jazzy
  ros-jazzy-desktop                    0.11.0-1noble.20261006.040109
  ros-jazzy-pcl-ros                    2.6.5-...
...
== Interfaces de red ==
eno1             DOWN
wlp4s0           UP             192.168.20.161/24 ...
```

Ningún paquete instalado en este capítulo debe figurar como `NO INSTALADO`. En la sección de red:

- **Placa Ethernet libre:** anotar su nombre (en esta PC, `eno1`); se usa en el capítulo 5.
- **Choque de redes:** si alguna placa ya tiene una dirección `192.168.1.x` (por ejemplo, el Wi-Fi de una casa cuyo router usa esa red), choca con la red de fábrica del sensor (capítulo 5).

5. Conexión y puesta en marcha del sensor

5.1 Conexión física

El cable del sensor termina en el **módulo adaptador**:

Conector del módulo	Se conecta a
Entrada de alimentación (plug DC 3,5 × 1,35 mm)	Fuente de 12 V / 1 A
RJ45 (Ethernet)	Placa Ethernet libre de la PC
USB-C	PC, solo en modo serie (punto 5.7)

> **Polaridad de la fuente**
> El manual no especifica la polaridad del plug. Antes de usar una fuente por primera vez (y siempre que no sea la original): enchufarla **sin conectarla al sensor**, multímetro en tensión continua (escala de 20 V), punta roja **dentro** del plug y punta negra **afuera**. Tiene que marcar unos **+12 V** (centro positivo). Medición de la fuente original del laboratorio: pendiente de registrar.

Pasos:

1. Apoyar el sensor sobre su almohadilla de goma, en una superficie firme.
2. Cable Ethernet del módulo a la placa libre de la PC.
3. Fuente al módulo. El sensor empieza a girar solo. En frío puede tardar unos minutos en entregar puntos, porque primero se calienta.

5.2 Configuración de red

El sensor viene de fábrica con la IP `192.168.1.62` y envía sus datos a `192.168.1.2`, puerto UDP 6201. Por eso la placa de la PC conectada al sensor tiene que tener **exactamente** la IP `192.168.1.2`.

```
./scripts/red/configurar_red_lidar.sh eno1      # eno1 = nombre de la placa (capítulo 4.5)
```

El script:

1. Verifica que ninguna otra placa esté en la red `192.168.1.x` (choque de redes).
2. Crea un perfil de red llamado `lidar-l2` con IP fija `192.168.1.2/24` y **sin puerta de enlace**, para que internet siga saliendo por el Wi-Fi. El perfil se activa solo cada vez que se conecta el cable.
3. Si el firewall `ufw` está activo, abre el puerto UDP 6201 en esa placa (pide la contraseña).

Salida esperada (última línea): `eno1 UP 192.168.1.2/24`.

Lo mismo, a mano:

```
nmcli connection add type ethernet con-name lidar-l2 ifname eno1
nmcli connection modify lidar-l2 connection.autoconnect yes connection.autoconnect-priority 10 \
    ipv4.method manual ipv4.addresses 192.168.1.2/24 ipv4.gateway "" ipv4.never-default yes \
    ipv6.method disabled
nmcli connection up lidar-l2
sudo ufw allow in on eno1 to any port 6201 proto udp comment "LiDAR Unitree L2"
```

Para deshacerlo: `nmcli connection delete lidar-l2` y `sudo ufw delete allow in on eno1 to any port 6201 proto udp`.

> **Choque de redes**
> Si el Wi-Fi (u otra placa) ya está en una red `192.168.1.x`, los datos del sensor no llegan bien. En ese caso hay que cambiar la red del sensor con el SDK (función `setLidarIpAddressConfig`; procedimiento todavía no probado en este proyecto).

5.3 Verificar la comunicación

¿El sensor está en la red?

```
ping -c 4 192.168.1.62
```

```
64 bytes from 192.168.1.62: icmp_seq=1 ttl=128 time=0.171 ms
...
4 packets transmitted, 4 received, 0% packet loss
```

¿Llegan los datos? El `ping` no alcanza: hay que ver que lleguen paquetes al puerto 6201. El script del proyecto solo escucha (no le envía nada al sensor) y no necesita `sudo`:

```
./scripts/red/verificar_udp_lidar.py
```

```
Escuchando 192.168.1.2:6201 durante 5 s...
segundo  paquetes por tamaño (bytes: cantidad)
      0  80: 240, 1044: 204, 1124: 11
      1  80: 239, 1044: 204, 1124: 12
...
Origen: 192.168.1.62:6101
Tráfico promedio: 1.96 Mbit/s
```

Los paquetes de 1044 bytes (unos 205 por segundo) son los **puntos**; los de 80 bytes (unos 240 por segundo), la **IMU**. Alternativa clásica: `sudo tcpdump -i eno1 udp port 6201 -c 10`.

5.4 SDK del sensor y programa de ejemplo

El SDK de Unitree se descarga **fuera** del repositorio del proyecto, en `~/lidar_ws/`:

```
mkdir -p ~/lidar_ws && cd ~/lidar_ws
git clone https://github.com/unitreerobotics/unilidar_sdk2.git
cd unilidar_sdk2
git checkout 0e3c51f512e6b8ff60b8c32f160b412cb48445c2      # versión probada (v2.0.10)
cd unitree_lidar_sdk && mkdir -p build && cd build
cmake .. && make -j"$(nproc)"
```

Termina con `[100%] Built target ...`. El aviso de CMake `Compatibility with CMake < 3.5 will be removed` no afecta. El mismo SDK se puede bajar como `.zip` desde la página de Unitree: es idéntico a esta versión.

> **No ejecutar set_ip_address**
> Junto con el ejemplo se compilan `set_ip_address` y `set_to_serial_mode`, que tienen IPs fijas de otra red (`192.168.123.110` / `.120`). **`set_ip_address` cambia la IP del sensor**, que deja de responder en `192.168.1.62`.

Ejecutar el ejemplo (se corta con **Ctrl+C**):

```
cd ~/lidar_ws/unilidar_sdk2/unitree_lidar_sdk/bin
./example_lidar_udp
```

```
Unilidar initialization succeed!
set Lidar work mode to: 0
lidar hardware version = 2.2.1.1
lidar firmware version = 2.8.11.1
lidar sdk version = 2.0.9
...
dirty percentage = 4.806547 %
An IMU msg is parsed!
    linear_acceleration (x, y, z) = [-1.0475, -0.3436, 10.4363]
...
A Cloud msg is parsed!
    cloud size  = 5112, ringNum = 18
```

- **hardware / firmware version:** las versiones del sensor (anotarlas).
- **dirty percentage:** suciedad estimada de la ventana; si es alta, limpiarla con un paño suave.
- **linear_acceleration:** con el sensor quieto, un eje marca cerca de 9,8 m/s² (la gravedad).
- **cloud size / ringNum:** unos 5.000 puntos por mensaje, 18 anillos.

5.5 Estado interno del sensor

Cada paquete de puntos trae también el **estado interno** del sensor. La herramienta del proyecto `estado_lidar` lo muestra (cerrar antes el ejemplo o el driver, que usan el mismo puerto):

```
cd ~/Documentos/LiDAR-RobotHumanoide/scripts/sensor
mkdir -p build && cd build
cmake .. && make
./estado_lidar udp 20        # 20 segundos
```

```
Resumen del estado (4325 paquetes de puntos):
                          mínimo     promedio      máximo
  sys_rotation_period    4621.000     4624.262     4629.000
  com_rotation_period  229426.000   230091.540   231579.000
  dirty_index               0.000        5.074       36.000
  packet_lost_up            0.000        0.000        0.000
  packet_lost_down          0.000        0.000        0.000
  apd_temperature          41.001       41.528       41.982
  apd_voltage             150.080      152.303      154.219
  laser_voltage            49.487       49.749       49.991
  imu_temperature          59.219       60.703       61.762
```

Campo	Interpretación
`sys_rotation_period`	Período del motor del **espejo** en µs: 1 / 4.621 µs ≈ **216 Hz**, igual al barrido vertical del manual
`com_rotation_period`	Período del motor del **cabezal** en µs: ≈ **4,35 Hz** (el manual indica 5,55 Hz de barrido horizontal; a consultar con Unitree)
`packet_lost_*`	Pérdida de paquetes internos: tiene que ser 0
`apd_temperature`, `apd_voltage`	Fotodetector: la tensión sube con la temperatura (compensación térmica)

Campo	Interpretación
`imu_temperature`	La IMU tiene calefacción controlada (~60 °C)

Lo importante es que los períodos de los motores sean **estables**: en el sensor del laboratorio varían menos de ±0,2 % (espejo) y ±0,7 % (cabezal).

5.6 Vibración del sensor

Con el driver corriendo (capítulo 6) y el sensor quieto sobre su almohadilla, la IMU permite medir la vibración:

```
./scripts/sensor/medir_vibracion_imu.py --segundos 20 --csv docs/mediciones/imu_reposo_<fecha>.csv
```

Muestra la aceleración media, su variación, la velocidad angular y las frecuencias dominantes de la vibración en cada eje. Con el sensor quieto, la velocidad angular tiene que estar cerca de 0 (si no, la medición no es válida).

> **Observación sobre el sensor del laboratorio**
>
> El sensor del laboratorio hace un ruido notorio al girar. La medición mostró una vibración vertical de 9,2 m/s² pico a pico en **8,7 Hz, el doble exacto de la rotación del cabezal**, y una aceleración media en reposo de 7,8 m/s² en lugar de 9,8. La vibración está ligada a la parte que gira; no hay un valor de referencia de un L2 sano para comparar. Igualmente, el sensor funciona y los mapas obtenidos son buenos. Los datos están en `docs/mediciones/` para una consulta con Unitree.

5.7 Conexión por USB (pendiente de probar)

El L2 también puede enviar sus datos por un puerto serie, a través del USB-C del módulo. Hay un solo modo activo a la vez, guardado en el sensor; de fábrica viene en Ethernet.

- Para pasar a USB hay que comunicarse primero por Ethernet y cambiar el modo con la herramienta del proyecto `scripts/sensor/build/cambiar_modo_lidar serie`. Después, apagar y prender el sensor. Para volver: `cambiar_modo_lidar ethernet` (conectado por USB).
- El driver se lanza con `modo:=usb` (capítulo 6), que usa el modo de trabajo 8. Con el `launch` original de Unitree, usar USB sin cambiar también el modo de trabajo devuelve el sensor a Ethernet y se corta la conexión.

Este procedimiento está preparado pero **todavía no se probó** con el sensor. Para usar el sensor en el robot, tiene que quedar en **modo Ethernet**.

6. Driver de ROS 2 y visualización

El **driver** es el nodo de ROS 2 que recibe los datos del sensor y los publica en los tópicos `/unilidar/cloud` (nube) y `/unilidar/imu` (IMU). Viene dentro del SDK de Unitree.

6.1 Compilar el driver

```
source /opt/ros/jazzy/setup.bash
cd ~/lidar_ws/unilidar_sdk2/unitree_lidar_ros2
rosdep check --from-paths src --ignore-src      # verifica que no falte nada
colcon build
```

```
All system dependencies have been satisfied
Starting >>> unitree_lidar_ros2
--- stderr: unitree_lidar_ros2
CMake Warning (dev) at /usr/lib/x86_64-linux-gnu/cmake/pcl/Modules/FindFLANN.cmake:45 (find_package):
  Policy CMP0144 is not set: ...
---
Finished <<< unitree_lidar_ros2 [25.5s]
Summary: 1 package finished [25.8s]
```

Los avisos de CMake vienen de la librería PCL y no afectan.

> **No mover el paquete**
>
> El driver busca la librería del SDK con una ruta relativa (tres carpetas hacia arriba). Hay que compilarlo en su carpeta original, `~/lidar_ws/unilidar_sdk2/unitree_lidar_ros2`, y no moverlo. El README de Unitree dice `unilidar_sdk/...`: la carpeta real se llama `unilidar_sdk2`.

6.2 Lanzar el driver

El proyecto tiene su propio archivo de lanzamiento, `config/unitree_lidar_ros2/l2_driver.launch.py`, basado en el de Unitree, con estas mejoras:

- **`modo:=ethernet`** o **`modo:=usb`**: configura juntos los dos parámetros que dependen de la conexión (`initialize_type` y `work_mode`).
- **`lidar_ip:=…`**, **`local_ip:=…`**, **`serial_port:=…`**: cambiar direcciones sin editar archivos.
- **`rviz:=false`**: no abrir RViz (por ejemplo, al trabajar por SSH).

Como el driver es un proceso que queda corriendo, conviene lanzarlo dentro de **tmux**:

```
tmux new -s lidar
source /opt/ros/jazzy/setup.bash
source ~/lidar_ws/unilidar_sdk2/unitree_lidar_ros2/install/setup.bash
cd ~/Documentos/LiDAR-RobotHumanoide
ros2 launch config/unitree_lidar_ros2/l2_driver.launch.py modo:=ethernet rviz:=false
```

```
[INFO] [unitree_lidar_ros2_node-1]: process started with pid [8192]
[unitree_lidar_ros2_node-1] initialize_type_ = 2
[unitree_lidar_ros2_node-1] [UDPHandler] bind udp port success. port 6201.
```

> **tmux**
>
> `tmux` mantiene la terminal "viva" aunque se cierre la ventana o se corte una conexión SSH. Salir dejándolo corriendo: **Ctrl+b**, soltar, y después **d**. Volver: `tmux attach -t lidar`. Detener el driver: dentro de tmux, **Ctrl+C**. Si se lanzó en otra terminal y no se sabe en cuál: `pkill -INT -f l2_driver.launch.py`.

6.3 Verificar los tópicos

En otra terminal (con `source /opt/ros/jazzy/setup.bash`):

```
ros2 topic list
ros2 topic hz /unilidar/cloud
ros2 topic hz /unilidar/imu
```

Tópico	Frecuencia medida	Contenido
`/unilidar/cloud`	**12,0 Hz** (un mensaje cada 83 ms)	Nube `PointCloud2`, marco `unilidar_lidar`, ~4.400 a 5.100 puntos
`/unilidar/imu`	**251 Hz**	IMU, marco `unilidar_imu`
`/tf`	—	Orientación estimada por la IMU (`unilidar_imu_initial` → `unilidar_imu`)

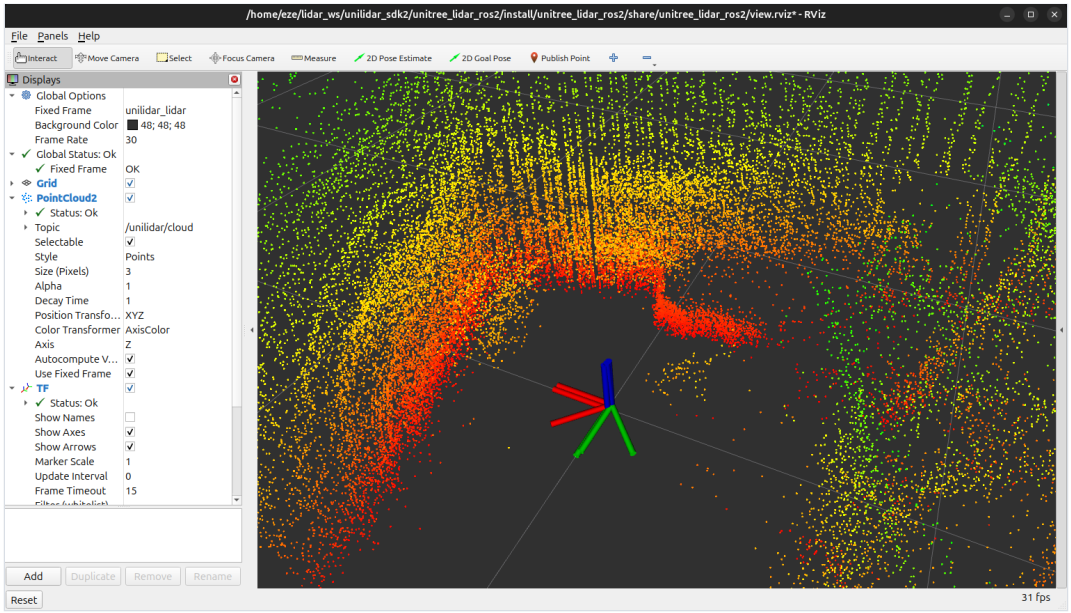
Cada punto de la nube tiene los campos `x`, `y`, `z` (metros), `intensity`, `ring` (anillo, de 0 a 17) y `time` (tiempo del punto dentro del barrido, en segundos), que son los que necesita el SLAM. Se pueden ver con `ros2 topic echo --once /unilidar/cloud --field fields`.

6.4 Ver la nube en RViz

RViz es una ventana gráfica: se abre en una **terminal del escritorio**. Con el driver corriendo:

```
source /opt/ros/jazzy/setup.bash
source ~/lidar_ws/unilidar_sdk2/unitree_lidar_ros2/install/setup.bash
rviz2 -d "$(ros2 pkg prefix unitree_lidar_ros2)/share/unitree_lidar_ros2/view.rviz"
```

`ros2 pkg prefix` devuelve dónde quedó instalado el driver, así el comando funciona en cualquier PC. La configuración de Unitree muestra la nube coloreada por altura en el marco `unilidar_lidar`.



Primera nube de puntos del L2 en RViz. El sensor apoyado en el piso ve las paredes alrededor y hacia arriba (laboratorio, 12/11/2026)

RViz y VS Code instalado como snap

Si RViz se lanza desde la terminal integrada de **VS Code instalado como snap**, falla con `symbol lookup error: /snap/core20/.../libpthread.so.0: undefined symbol: __libc_pthread_init`. La terminal de VS Code agrega variables de entorno propias del snap. Solución: abrir RViz desde la terminal de Ubuntu.

7. Grabación de datos

Grabar permite generar el mapa **después**, en la PC, sin repetir el recorrido. Así va a trabajar el robot: graba mientras se lo maneja y el mapa se calcula luego.

7.1 Grabar

Con el driver corriendo (capítulo 6), en otra terminal:

```
source /opt/ros/jazzy/setup.bash
cd ~/Documentos/LiDAR-RobotHumanoide
./scripts/grabacion/grabar_lidar.sh <nombre> [segundos] ["descripción"]
```

- **`<nombre>`:** letras, números y guiones bajos (por ejemplo `laboratorio_vuelta1`).
- **`segundos`:** opcional. Sin duración, graba hasta cortar con **Ctrl+C**. Para dar una descripción sin duración, dejar la duración vacía: `""`.
- **`"descripción"`:** opcional; queda guardada junto a la grabación.

Ejemplo:

```
./scripts/grabacion/grabar_lidar.sh sala_quieta 90 "Sensor en el piso, nadie se mueve"
```

El script:

1. **Verifica** que lleguen mensajes de la nube y de la IMU y que queden al menos 5 GB libres.
2. **Graba** `/unilidar/cloud` y `/unilidar/imu` en formato MCAP en `bags/<fecha>_<hora>_<nombre>/`.
3. **Vigila** que sigan llegando datos: si pasan más de 1,5 s sin nubes (un cable que se soltó), avisa en pantalla con un pitido cada 2 s.
4. **Corta** al cumplirse la duración o con Ctrl+C, cerrando el archivo de forma ordenada.
5. Guarda un `descripcion.txt` y muestra el resumen.

```
Files:             2026-11-12_1001_prueba_script_0.mcap
Bag size:          28.2 MiB
Storage id:        mcap
Duration:          14.438826328s
Messages:          3792
Topic information: Topic: /unilidar/cloud | Type: sensor_msgs/msg/PointCloud2 | Count: 173 | ...
                   Topic: /unilidar/imu | Type: sensor_msgs/msg/Imu | Count: 3619 | ...
```

Comprobación rápida: 173 nubes / 14,44 s = 12 por segundo, y 3619 IMU / 14,44 s = 250 por segundo: no se perdieron mensajes.

> **Terminar bien una grabación**
>
> Después del Ctrl+C, **esperar a que aparezca el resumen** (`Files:`, `Duration:`, `Messages:`) antes de desconectar el sensor. Si se lo desconecta antes, la grabación queda abierta sin datos. En ese caso la terminal empieza a pitar con el aviso `NO LLEGAN DATOS DEL LiDAR`.

Lo mismo, a mano (sin controles ni aviso):

```
ros2 bag record -s mcap -o bags/mi_grabacion --topics /unilidar/cloud /unilidar/imu
```

Duración	Tamaño aproximado
1 minuto	117 MB
10 minutos	1,2 GB

Las grabaciones **no se suben a git** (la carpeta `bags/` está excluida). Cada grabación útil se anota en `docs/grabaciones.md`.

7.2 Revisar una grabación

```
./scripts/grabacion/revisar_grabacion.py bags/<carpeta>
```

Revisa tres cosas:

- **Huecos:** intervalos entre mensajes mayores a 3 veces el normal.
- **Corte al final:** si el sensor dejó de enviar datos antes de que se cortara la grabación.
- **Arranque:** si el sensor estaba **quieto** en los primeros 4 s. Es importante para el SLAM (capítulo 8).

Ejemplo real, una grabación completa y con arranque quieto:

```
/unilidar/cloud
  mensajes: 874 en 72.78 s -> 12.01 por segundo
  intervalo medio: 83.4 ms | máximo: 88.5 ms
  sin huecos
...
Duración de la grabación (descripcion.txt): 74 s | último dato a los 74 s (14:43:58)

Resultado: grabación completa

Arranque (primeros 4 s de IMU, 1005 muestras):
  aceleración media [m/s²]: x=0.07 y=-0.09 z=8.95 (módulo 8.95)
  gravedad sobre el eje z del sensor, a 0.7° de ese eje
  desvío del giróscopo [rad/s]: x=0.051 y=0.040 z=0.050 -> QUIETO (límite 0.1 rad/s)
```

Y una grabación en la que se desconectó el sensor creyendo que ya se había cortado:

```
Duración de la grabación (descripcion.txt): 271 s | último dato a los 16 s (14:32:48)
  ATENCIÓN: no llegaron datos durante los últimos 255 s. El sensor dejó de enviar datos (¿cable
desconectado o sin alimentación?).
Resultado: GRABACIÓN INCOMPLETA: revisar los avisos de arriba
```

7.3 Ver una grabación en RViz

No hace falta el sensor, pero **el driver tiene que estar detenido** (si no, se mezclan los datos en vivo con los grabados).

> **Ver el lugar más completo**
>
> Cada nube cubre solo una parte del barrido. En RViz, dentro de *PointCloud2*, subir **Decay Time** de 1 a 10: se acumulan los últimos 10 segundos de puntos. Sirve solo con el sensor quieto; si se movió, los puntos acumulados quedan corridos. Para eso está el SLAM.

En dos terminales del escritorio:

```
# Terminal 1: reproducir (--loop la repite hasta Ctrl+C; --rate 0.5 la hace más lenta)
source /opt/ros/jazzy/setup.bash
cd ~/Documentos/LiDAR-RobotHumanoide
ros2 bag play bags/<carpeta> --loop

# Terminal 2: RViz
source /opt/ros/jazzy/setup.bash
source ~/lidar_ws/unilidar_sdk2/unitree_lidar_ros2/install/setup.bash
rviz2 -d "$(ros2 pkg prefix unitree_lidar_ros2)/share/unitree_lidar_ros2/view.rviz"
```

8. SLAM y generación del mapa 3D

El SLAM **Point-LIO** procesa una grabación y genera el **mapa 3D** del lugar (archivo `.pcd`) y la **trayectoria** del sensor. No hace falta el sensor conectado.

8.1 Descargar y compilar Point-LIO

Se usa la versión para ROS 2 `dfloreaa/point_lio_ros2`, que trae la configuración del L2. Se compila en un espacio de trabajo propio:

```
source /opt/ros/jazzy/setup.bash
mkdir -p ~/lidar_ws/point_lio_ws/src && cd ~/lidar_ws/point_lio_ws/src
git clone https://github.com/dfloreaa/point_lio_ros2.git
cd point_lio_ros2 && git checkout a8e2d0d5090af97ead8dd4fac3d37cf3dbb33ff7     # versión probada
cd ~/lidar_ws/point_lio_ws
rosdep check --from-paths src --ignore-src
colcon build --cmake-args -DCMAKE_BUILD_TYPE=Release
```

```
All system dependencies have been satisfied
Starting >>> point_lio
...
Finished <<< point_lio [52.8s]
Summary: 1 package finished [53.0s]
```

Los avisos de CMake (`CMP0144`, `CMP0148`) no afectan.

8.2 Configuración para el L2

Se usa la configuración que trae el paquete, sin cambios (`config/unilidar_l2.yaml`). Los parámetros que más influyen en el uso:

Parámetro	Valor	Significado
`lid_topic`, `imu_topic`	`/unilidar/cloud`, `/unilidar/imu`	Tópicos del driver del L2
`blind`	0,5 m	Descarta puntos a menos de 0,5 m: la mano o el cuerpo de quien lleva el sensor
`start_in_aggressive_motion`	false	Hay que **empezar con el sensor quieto**
`gravity_align`	true	Alinea el eje Z del mapa con la gravedad medida al empezar, **en cualquier orientación del sensor**
`pcd_save_en`	true	Guarda el mapa al cerrar Point-LIO

> **Arrancar quieto**
>
> Point-LIO mide la dirección de la gravedad en los primeros instantes y con eso fija la orientación de **todo** el mapa. Si en ese momento el sensor se mueve, el mapa queda **inclinado**. En una prueba con el sensor en la mano desde el inicio, el mapa quedó inclinado 13,5° y el error de cierre fue de 1,06 m. Repitiendo el mismo recorrido con el sensor **apoyado y quieto** al empezar, la inclinación bajó a 1,9° y el error de cierre a 10,5 cm.

8.3 Generar el mapa de una grabación

Con el driver **detenido**:

```
./scripts/mapas/procesar_grabacion.sh bags/<carpeta> [nombre_del_mapa] [--rviz]
```

El script lanza Point-LIO, reproduce la grabación una vez, registra la trayectoria, cierra Point-LIO (que es cuando guarda el mapa) y deja todo en una carpeta propia. Tarda lo mismo que duró la grabación. Con `--rviz` se ve cómo se arma el mapa.

```
Grabación: bags/2026-11-12_1028_persona_caminando
Carpeta del mapa: .../mapas/2026-11-12_1412_persona_caminando

Point-LIO en marcha. Reproduciendo la grabación (tarda lo mismo que duró)...
Reproducción terminada. Deteniendo Point-LIO y guardando el mapa...

Mapa: 2026-11-12_1412_persona_caminando/mapa.pcd
Puntos: 1128052
Tamaño: 35M
Trayectoria: 588 muestras, 59.1 s
Largo del recorrido: 4.45 m
Zona recorrida (x, y, z): 0.06 x 0.11 x 0.02 m
Error de cierre (final vs. inicio): 0.052 m (horizontal 0.052 m, vertical -0.002 m)

Para verlo:    ./scripts/mapas/ver_mapa.sh mapas/2026-11-12_1412_persona_caminando
Para medirlo:  ./scripts/mapas/analizar_mapa.py mapas/2026-11-12_1412_persona_caminando
```

Cada mapa queda en `mapas/<fecha>_<nombre>/`:

Archivo	Contenido	¿En git?
`mapa.pcd`	El mapa 3D	No (pesa decenas de MB)
`resumen.txt`	Datos del mapa y de la trayectoria	Sí
`trayectoria.csv`	Posición estimada del sensor cada 0,1 s	Sí
`pointlio.log`	Registro de Point-LIO	No

Indicadores de calidad del resumen:

- **Zona recorrida:** tiene que parecerse a lo que se caminó. (En el ejemplo, el sensor estaba fijo: 6 × 11 cm.)
- **Error de cierre:** distancia entre la posición final y la inicial. Si se terminó en el punto de partida, debería ser de pocos centímetros. Con el sensor quieto da 5 a 9 cm.
- **Largo del recorrido:** solo orientativo; incluye el pequeño temblor de la estimación.

8.4 Lo mismo, a mano

Con el driver detenido, en tres terminales con el entorno cargado (`source /opt/ros/jazzy/setup.bash` y `source ~/lidar_ws/point_lio_ws/install/setup.bash`):

1. **Terminal 1**: `ros2 launch point_lio mapping_unilidar_l2.launch.py rviz:=false` y esperar el mensaje `... file opened`.
2. **Terminal 2** (opcional): `./scripts/mapas/registrar_trayectoria.py /tmp/trayectoria.csv`.
3. **Terminal 3**: `ros2 bag play bags/<carpeta>`.

4. Al terminar la reproducción: **Ctrl+C en la terminal 2** (muestra el resumen de la trayectoria) y después **Ctrl+C en la terminal 1**. Point-LIO guarda el mapa en `~/lidar_ws/point_lio_ws/src/point_lio_ros2/PCD/scans.pcd`.
5. Copiar el mapa con un nombre propio, porque la próxima corrida lo **sobrescribe**:

```
CARPETA=mapas/$(date +%Y-%m-%d_%H%M)_<nombre>
mkdir -p $CARPETA
cp ~/lidar_ws/point_lio_ws/src/point_lio_ros2/PCD/scans.pcd $CARPETA/mapa.pcd
cp /tmp/trayectoria.csv $CARPETA/trayectoria.csv
```

8.5 Ver y medir el mapa

Desde una terminal del escritorio:

```
./scripts/mapas/ver_mapa.sh mapas/<carpeta_del_mapa>
```

Muestra el mapa en RViz y el **recorrido** del sensor: línea blanca, con una **esfera verde** en la partida y una **roja** en la llegada. Que las paredes crucen la grilla en diagonal es normal (los ejes los fija la dirección del sensor al empezar); lo que no es normal es que el piso no sea paralelo a la grilla.

Para ver las medidas con números:

```
./scripts/mapas/analizar_mapa.py mapas/<carpeta_del_mapa>
```

```
  eje   mínimo   máximo  |  útil (1 %-99 %)   extensión útil
   x    -3.57     5.68  |   -3.35 a   3.47     6.82 m
...
  Planos principales (muestra de 40000 puntos):
    plano horizontal a z =  1.95 m: inclinación  1.9° (20 % de los puntos)
...
Trayectoria: 720 muestras, 72.5 s
  error de cierre: 0.105 m (horizontal 0.105 m, vertical +0.006 m)
```

La **extensión útil** son las medidas aproximadas del lugar. La **inclinación** de los planos horizontales (piso y techo) indica si el mapa quedó nivelado: tiene que ser de 1 a 2° como máximo. Si es mayor, la causa más común es haber arrancado con el sensor en movimiento. El capítulo 10 explica cómo nivelar el mapa.

9. Mapeo de una habitación

Este capítulo reúne, en orden, todo lo necesario para obtener el mapa de un lugar recorriéndolo con el sensor en la mano.

9.1 Preparación

1. **Conectar** el sensor (Ethernet y fuente) y dejarlo encendido unos minutos: en frío tarda en entregar puntos.
2. **Verificar la red:** `ip -br addr show eno1` tiene que mostrar `UP 192.168.1.2/24`.
3. **Lanzar el driver** dentro de tmux (capítulo 6) y verificar con `ros2 topic hz /unilidar/cloud` que da unos 12 Hz.
4. **Preparar el lugar:** abrir las puertas que se van a atravesar, despejar el camino y, si es posible, que no haya gente moviéndose.
5. **Cables:** la fuente es de pared y los datos van por cable a la notebook. Usar un alargue, llevar la notebook encima y dejar el cable con juego para que no se tense al levantar el sensor.

9.2 Cómo llevar el sensor

Point-LIO funciona con el sensor en cualquier orientación, siempre que **empiece quieto**. Lo que cambia es qué parte del lugar queda en el mapa:

Orientación	Cómo	Queda en el mapa	No queda
Horizontal	Cúpula hacia arriba, a la altura de la cabeza o más arriba	Paredes, objetos altos y techo, en 360°	Piso y obstáculos bajos
Vertical	Cúpula hacia adelante, a la altura del pecho	Piso, obstáculos bajos, paredes y techo de adelante	Lo que queda detrás

Para el mapa 2D que va a usar el robot hace falta ver el piso y los obstáculos bajos: conviene la orientación **vertical**. En los dos casos, no tapar la ventana del sensor con la mano ni con el cable.

9.3 Grabar el recorrido

```
./scripts/grabacion/grabar_lidar.sh habitacion_vertical "" "Habitación, sensor vertical, inicio y fin en la puerta"
```

1. **Sensor apoyado y quieto 10 s** sobre una mesa, un estante o el piso. Puede estar en otra orientación que la de trabajo (por ejemplo, horizontal sobre la mesa): lo importante es que esté **quieto**. Recién después, levantarlo.
2. Caminar **despacio** y girar **suave**: los giros bruscos son la causa más común de mapas deformados. En las puertas, ir más despacio, porque hay pocas referencias.
3. **Volver al punto de partida**, apoyarlo y dejarlo quieto otros 10 s.
4. **Ctrl+C** y esperar el resumen antes de desconectar nada.

9.4 Generar, revisar y ver el mapa

```
tmux attach -t lidar                                  # y Ctrl+C: detener el driver
./scripts/grabacion/revisar_grabacion.py bags/<carpeta>  # "grabación completa" y arranque "QUIETO"
./scripts/mapas/procesar_grabacion.sh bags/<carpeta>     # genera mapas/<fecha>_<nombre>/
./scripts/mapas/ver_mapa.sh mapas/<carpeta_del_mapa>     # terminal del escritorio
```

Qué revisar:

- **Error de cierre** de pocos centímetros (si se volvió al punto de partida).
- **Inclinación** de piso y techo de 1–2° como máximo (`analizar_mapa.py`).
- En RViz: paredes simples, sin duplicados; piso paralelo a la grilla; esferas verde y roja juntas.

Si algo falla, repetir la grabación más despacio y arrancando bien quieto.

9.5 Resultado obtenido

Habitación recorrida con el sensor vertical, con un recorrido de unos 74 s:

Dato	Valor
Grabación	74 s, completa, arranque quieto
Mapa 3D	1,10 millones de puntos (34 MB)
Error de cierre	**10,5 cm**
Inclinación (antes de nivelar)	1,9°
Medidas	unos 3,2 × 7,9 m en planta y 2,85 m de alto

Las grabaciones y mapas de referencia del proyecto, con sus resultados, están registrados en `docs/grabaciones.md` y `docs/mapas.md`.

10. Procesamiento del mapa: mapa 3D limpio y mapa 2D

El mapa que genera Point-LIO se convierte en los dos productos que va a usar el robot. No hace falta el sensor.

10.1 Mapa 3D limpio

```
./scripts/mapas/limpiar_mapa.py mapas/<carpeta_del_mapa>
```

Paso	Qué hace	Parámetro (defecto)
Reducción de densidad	Un punto por cubo de 3 cm (Point-LIO repite muchas veces los mismos puntos)	`--voxel 0.03`
Puntos sueltos	Quita los puntos mucho más lejos de sus 10 vecinos que el promedio (ruido, reflejos)	`--vecinos 10`, `--sigma 2.0`
Nivelación	Busca el **piso** (el plano horizontal más bajo), lo deja horizontal y en **z = 0**, y afina con todos los puntos del piso	`--referencia piso`
Alineación	Gira el mapa en planta para que la pared principal quede paralela a un eje	`--sin-alinear` la desactiva

> **RANSAC**
> Para encontrar un plano (el piso, una pared) en una nube con ruido, se eligen 3 puntos al azar, se arma el plano que pasa por ellos y se cuentan los puntos cercanos. Se repite cientos de veces y se queda el plano que más puntos explica. Después se reajusta con todos sus puntos.

Salida real (habitación del capítulo 9):

```
1. Reducción a celdas de 3 cm: 1.102.546 -> 350.797 puntos
2. Puntos sueltos quitados: 11.446 (vecinos=10, sigma=2.0) -> quedan 339.351
   plano horizontal a z =   1.96 m, inclinación  1.3°,  8.9 % de los puntos
   plano horizontal a z =  -0.89 m, inclinación  1.9°,  4.7 % de los puntos
   plano horizontal a z =   0.00 m, inclinación  0.6°,  3.4 % de los puntos
...
3. Nivelación con el piso: inclinación corregida 1.9°; piso llevado a z = 0 (desplazamiento +0.88 m)
   Refinamiento con la franja del piso (12.395 puntos): 0.35° adicionales
4. Alineación de paredes: giro de +40.0° alrededor del eje vertical
5. Guardado .../mapa_limpio.pcd: 339.351 puntos, 5.4 MB
   Extensión útil (1 %-99 %): x 3.21 m, y 7.88 m, z -0.06 a 2.85 m
```

El plano a −0,89 m es el piso; el de 0,00 m, la **mesa** donde se apoyó el sensor al empezar (el origen del mapa es la posición inicial del sensor). Resultado: piso en z = 0, inclinación de 0,3 a 0,8° y un archivo de 5,4 MB en lugar de 34 MB. Se guardan además `trayectoria_limpia.csv`, `transformacion.txt` (la matriz aplicada, que relaciona las coordenadas de Point-LIO con las del mapa limpio) y `limpieza.txt`.

> **Sirve para cualquier mapa que vea el piso**
>
> El script funciona con cualquier `.pcd`, de cualquier lugar. Para nivelar necesita **ver el piso**. Si el plano más bajo no puede ser el piso (tiene más del 5 % de los puntos por debajo), no nivela y lo avisa: en ese caso usar `--referencia techo` o `--sin-nivelar`.

Verificación: `./scripts/mapas/analizar_mapa.py mapas/<carpeta>/mapa_limpio.pcd` (inclinación de 1° o menos), y visualmente: `./scripts/mapas/ver_mapa.sh mapas/<carpeta>/mapa_limpio.pcd mapas/<carpeta>/trayectoria_limpia.csv`.

10.2 Mapa 2D de ocupación para Nav2

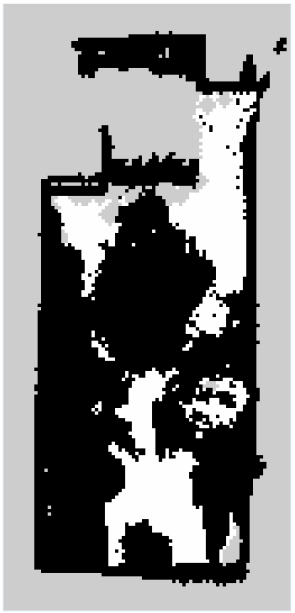
```
./scripts/mapas/generar_mapa_2d.py mapas/<carpeta_del_mapa>
```

Usa `mapa_limpio.pcd` (hay que hacer antes el punto 10.1). Arma una grilla de 5 cm vista desde arriba y clasifica cada celda:

- **Ocupada (negro):** tiene al menos 2 puntos en la **franja de alturas** de 0,10 a 1,80 m sobre el piso, donde el robot puede chocar. El piso y lo que está por encima del robot no cuentan.
- **Libre (blanco):** se vio el piso y no hay obstáculos, o pasó por ahí quien llevaba el sensor (unos 20 cm alrededor de la trayectoria). Se rellenan huecos chicos.
- **Desconocida (gris):** no se vio.

Parámetro	Defecto	Significado
`--resolucion`	0.05	Tamaño de celda (m)
`--altura-min` / `--altura-max`	0.10 / 1.80	Franja de obstáculos (m). `--altura-max` debe ser al menos la altura del robot
`--min-puntos`	2	Puntos para considerar ocupada una celda
`--piso`	0.05	Tolerancia del piso (m)

```
Grilla: 85 x 180 celdas de 5 cm = 4.25 x 9.00 m
Origen (esquina inferior izquierda): x = -2.497 m, y = -3.800 m
Franja de obstáculos: 0.10 a 1.80 m sobre el piso
Celdas: ocupadas 41.7 %, libres 14.7 %, desconocidas 43.6 %
```



Mapa 2D de la habitación. Negro: ocupado (paredes y muebles); blanco: libre; gris: desconocido

Las manchas negras dentro de la habitación son **muebles**: hay unos 5 m² de superficies entre 0,7 y 1,0 m de altura (mesas y escritorios). Para un robot, una mesa es un obstáculo.

El script genera `mapa_2d.pgm` (la imagen), `mapa_2d.png` (copia para ver) y `mapa_2d.yaml`:

```
image: mapa_2d.pgm
mode: trinary
resolution: 0.05
origin: [-2.497, -3.800, 0.0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.196
```

> **El umbral free_thresh**
>
> Nav2 calcula, para cada píxel, `p = (255 − valor) / 255`. Las celdas desconocidas (gris, 205) dan p = 0,196. Con `free_thresh: 0.196` quedan **desconocidas**; con un valor mayor (por ejemplo 0,25) pasarían a ser **libres**, y el robot creería que puede ir por zonas que nunca se vieron.

10.3 Verificar el mapa 2D con Nav2

```
./scripts/mapas/ver_mapa_2d.sh mapas/<carpeta_del_mapa>       # terminal del escritorio
```

Carga el mapa con `nav2_map_server` (el mismo programa que va a usar el robot), lo activa y lo muestra en RViz visto desde arriba. A mano:

```
ros2 run nav2_map_server map_server --ros-args -p yaml_filename:=<ruta completa>/mapa_2d.yaml
ros2 run nav2_util lifecycle_bringup map_server      # en otra terminal: lo activa
rviz2 -d ~/Documentos/LiDAR-RobotHumanoide/config/rviz/ver_mapa_2d.rviz
```

> **Nodos con ciclo de vida**
>
> Los nodos de Nav2 arrancan **inactivos** y solo publican después de pasar por los estados *configure* y *activate*. `lifecycle_bringup` hace esas dos transiciones. Sin eso, RViz muestra "No map received".

Para comprobar cómo interpreta Nav2 cada celda (con el mapa cargado y activado):

```
./scripts/mapas/verificar_mapa_2d.py
```

```
Mapa en /map: 85 x 180 celdas de 5 cm (4.25 x 9.00 m), origen (-2.497, -3.800), marco 'map'
Nav2 recibe: ocupadas (100) 41.7 %, libres (0) 14.7 %, desconocidas (-1) 43.6 %, otros valores 0.0 %
```

Los porcentajes tienen que ser **iguales** a los de `generar_mapa_2d.py`.

11. Problemas frecuentes

Los problemas encontrados durante el desarrollo, con el mensaje de error literal cuando lo hay.

11.1 Instalación y repositorio

Síntoma	Causa	Solución
`fatal: could not read Username for 'https://github.com'`	No hay credenciales de GitHub	`gh auth login` y `gh auth setup-git` (capítulo 4.4)
`E: Unable to locate package ros-jazzy-desktop`	Falta el repositorio de ROS 2 o un `apt update`	Repetir el capítulo 4.2
`apt update`: `Conflicting values set for option Signed-By`	El repositorio de ROS 2 se agregó dos veces, con el método viejo (`ros2.list`) y el nuevo	`sudo rm /etc/apt/sources.list.d/ros2.list` y `sudo apt update`
`ros2: orden no encontrada`	La terminal no cargó ROS 2	`source /opt/ros/jazzy/setup.bash` o abrir una terminal nueva
`sudo rosdep init`: `ERROR: default sources list file already exists`	Ya estaba inicializado	Ignorarlo y correr `rosdep update`

11.2 Red y sensor

Síntoma	Causa	Solución
`ping 192.168.1.62` sin respuesta; `eno1` en `DOWN`	Sin enlace: cable, alimentación o placa equivocada	Revisar fuente y cable; `cat /sys/class/net/eno1/carrier` debe dar `1`
`ping` responde pero no llegan datos	Firewall, o la PC no tiene exactamente `192.168.1.2`	`ip -br addr`; regla de `ufw` del capítulo 5.2
`[Errno 98] Address already in use`	Otro programa usa el puerto 6201 (driver, ejemplo, `estado_lidar`)	Cerrarlo antes
`example_lidar_udp` no muestra nada	Espera datos sin límite de tiempo: no llegan	Verificar con `verificar_udp_lidar.py`
El sensor gira pero hay pocos puntos	Arranque en frío	Esperar unos minutos
El sensor dejó de responder en `192.168.1.62`	Se ejecutó `set_ip_address`, o está en modo serie	Probar `ping 192.168.123.110`; ver capítulo 5.7
Ruido fuerte al girar	Ver la observación del capítulo 5.6	Medir con `medir_vibracion_imu.py` y `estado_lidar`

Cuándo se cortó el enlace. Si en algún momento dejan de llegar datos:

```
ip -br addr show eno1                      # debe decir UP y 192.168.1.2/24
journalctl -k --since "-30 min" --no-pager | grep -i "eno1"
```

```
nov 12 14:32:48 eze-laptop kernel: r8169 0000:03:00.0 eno1: Link is Down
```

`Link is Down` indica que se perdió el enlace físico (cable suelto o sensor sin alimentación). Al reconectar, la red se reconfigura sola y el driver vuelve a recibir datos sin reiniciarlo.

11.3 Driver y RViz

Síntoma	Causa	Solución
`rviz2: symbol lookup error: /snap/core20/... undefined symbol: __libc_pthread_init`	RViz lanzado desde la terminal de VS Code instalado como snap	Abrirlo desde la terminal de Ubuntu
`PackageNotFoundError: "package 'unitree_lidar_ros2' not found`	Falta cargar el entorno del driver	`source ~/lidar_ws/unilidar_sdk2/unitree_lidar_ros2/install/setup.bash`
Error al compilar: no encuentra **`libunilidar_sdk2.a`**	Se movió el paquete del driver	Compilarlo en su carpeta original
En RViz: **`Fixed Frame [...] does not exist`**	El marco fijo no coincide	*Global Options* → *Fixed Frame*: `unilidar_lidar` (nube) o `map` (mapas)
Por USB, se corta la conexión al lanzar el driver	Se usó el modo serie con `work_mode: 0`	Usar `modo:=usb` del launch del proyecto

11.4 Grabación

Síntoma	Causa	Solución
`ERROR: no llegan mensajes en /unilidar/cloud. ¿Está corriendo el driver?`	El driver no está corriendo, o se cortó el enlace	Lanzar el driver; revisar la red (11.2)
Pitidos y **`NO LLEGAN DATOS DEL LiDAR`** durante la grabación	Cable suelto, o se desconectó el sensor sin cortar la grabación	Revisar conexiones; si ya se terminó, Ctrl+C y esperar el resumen
`revisar_grabacion.py`: **`GRABACIÓN INCOMPLETA`**	Se cortaron los datos antes de terminar la grabación	Repetir la grabación
`revisar_grabacion.py`: arranque **`EN MOVIMIENTO`**	El sensor no estaba quieto al empezar	Repetir arrancando con el sensor apoyado
En RViz se ven dos nubes al reproducir	El driver sigue corriendo	Detenerlo antes de `ros2 bag play`

11.5 SLAM y mapas

Síntoma	Causa	Solución
`ERROR: ya hay algo publicando /unilidar/cloud`	El driver o un `ros2 bag play` siguen abiertos	Detenerlos antes de procesar
Mapa inclinado (planos horizontales de más de 2°)	Se arrancó con el sensor en movimiento	Repetir arrancando quieto; o nivelar con `limpiar_mapa.py`

Síntoma	Causa	Solución
Paredes duplicadas o mapa torcido	Giros bruscos o pocas referencias	Repetir más despacio
El mapa no tiene piso ni muebles bajos	Sensor horizontal	Grabar con el sensor vertical
`limpiar_mapa.py`: `Nivelación: NO se hace. ... no es el piso`	El mapa no ve el piso	`--referencia techo`, o grabar con el sensor vertical
El mapa 2D sale casi todo gris	No se ve el piso o el mapa no se niveló	Revisar `limpieza.txt` y la orientación del sensor
RViz: "No map received" (mapa 2D)	El servidor de mapas no se activó	`ros2 run nav2_util lifecycle_bringup map_server`
`verificar_mapa_2d.py`: porcentajes distintos u "otros valores"	Umbrales del `.yaml` modificados	`occupied_thresh: 0.65`, `free_thresh: 0.196`

Anexo A. Parámetros

A.1 Launch del driver (`config/unitree_lidar_ros2/l2_driver.launch.py`)

Argumento	Defecto	Significado
`modo`	`ethernet`	`ethernet` (`initialize_type` 2, `work_mode` 0) o `usb` (`initialize_type` 1, `work_mode` 8)
`lidar_ip`	`192.168.1.62`	IP del sensor (modo Ethernet)
`local_ip`	`192.168.1.2`	IP de la PC en la placa del sensor
`serial_port`	`/dev/ttyACM0`	Puerto serie (modo USB)
`rviz`	`true`	Abrir RViz

Parámetros fijos del driver: `cloud_topic` `unilidar/cloud` (marco `unilidar_lidar`), `imu_topic` `unilidar/imu` (marco `unilidar_imu`), puertos UDP 6101 → 6201, `baudrate` 4000000, `cloud_scan_num` 18, `range_min` 0, `range_max` 100 m, `use_system_timestamp` true.

A.2 Point-LIO (`config/unilidar_l2.yaml` y `mapping_unilidar_l2.launch.py`)

Parámetro	Valor	Significado
`lidar_type` / **`scan_line`**	5 / 18	Formato de nube de Unitree, 18 anillos
`timestamp_unit`	0	Tiempo de cada punto en segundos
`blind`	0,5 m	Distancia mínima de los puntos usados
`imu_en` / **`acc_norm`**	true / 9,81	Usa la IMU; aceleración en m/s²
`start_in_aggressive_motion`	false	Arranque con el sensor quieto
`gravity_align`	true	Alinea el mapa con la gravedad inicial
`extrinsic_est_en`	false	No estima la posición IMU–LiDAR (usa la de fábrica)
`extrinsic_T`	`[0.007698, 0.014655, -0.00667]`	Posición de la IMU respecto de la nube (m)
`filter_size_surf` / **`filter_size_map`**	0,1 / 0,1 m	Resolución de la nube procesada y del mapa interno
`pcd_save_en` / **`interval`**	true / −1	Guarda todo el mapa en un único archivo al cerrar

A.3 Scripts del proyecto

Script	Argumentos y opciones
`red/configurar_red_lidar.sh`	`<placa> [ip/máscara]` (defecto `192.168.1.2/24`)
`red/verificar_udp_lidar.py`	`--ip`, `--puerto` (6201), `--segundos` (5)

Script	Argumentos y opciones
sensor/build/estado_lidar	udp [segundos] [ip_lidar] [ip_pc] o serie [segundos] [puerto]
sensor/build/cambiar_modo_lidar	serie [ip_lidar] [ip_pc] o ethernet [puerto]
sensor/medir_vibracion_imu.py	--segundos (15), --csv <archivo>, --topico
grabacion/grabar_lidar.sh	<nombre> [segundos] ["descripción"]; variable BAGS_DIR
grabacion/revisar_grabacion.py	<carpeta_de_la_grabación>
mapas/procesar_grabacion.sh	<carpeta_de_la_grabación> [nombre] [--rviz]; variable POINT_LIO_WS
mapas/ver_mapa.sh	<carpeta_del_mapa ó .pcd> [trayectoria.csv]
mapas/analizar_mapa.py	<carpeta_del_mapa ó .pcd> [trayectoria.csv]
mapas/limpiar_mapa.py	--voxel (0,03), --vecinos (10), --sigma (2,0), --referencia piso|techo, --sin-nivelar, --sin-alinear, --salida
mapas/generar_mapa_2d.py	--resolucion (0,05), --altura-min (0,10), --altura-max (1,80), --piso (0,05), --min-puntos (2), --cerrar (2), --margen (0,5)
mapas/ver_mapa_2d.sh	<carpeta_del_mapa ó mapa_2d.yaml>

Las rutas son relativas a `scripts/`. Cada script explica su uso en el encabezado del archivo.

Anexo B. Versiones

Versiones con las que se verificó esta guía (noviembre de 2026). Si en el futuro una versión nueva no funciona, estas son las de referencia. Detalle actualizado en `docs/versiones.md`.

B.1 Sensor y repositorios

Elemento	Versión
Unitree L2: hardware / firmware	2.2.1.1 / 2.8.11.1
`unilidar_sdk2` (SDK y driver ROS 2)	Tag `v2.0.10`, commit `0e3c51f512e6b8ff60b8c32f160b412cb48445c2`
`point_lio_ros2` (dfloreaa)	Commit `a8e2d0d5090af97ead8dd4fac3d37cf3dbb33ff7` (2025-08-14)

B.2 Sistema

Elemento	Versión
Ubuntu	24.04.5 LTS, kernel 7.0.0-42 (x86_64)
gcc / CMake / Python	13.3.0 / 3.28.3 / 3.12.3

B.3 Paquetes apt

Paquete	Versión
`ros-jazzy-desktop`	0.11.0-1noble.20261006.040109
`ros-dev-tools`	1.0.3
`python3-colcon-common-extensions`	0.3.0-100
`python3-rosdep`	0.27.0-1
`ros-jazzy-pcl-ros`	2.6.5-1noble.20260916.065720
`ros-jazzy-pcl-conversions`	2.6.5-1noble.20260912.165932
`libpcl-dev`	1.14.0+dfsg-1
`libeigen3-dev`	3.4.0-4build0.1
`ros-jazzy-rosbag2`, `ros-jazzy-rosbag2-storage-mcap`	0.26.11
`ros-jazzy-nav2-map-server`, `ros-jazzy-nav2-util`	1.3.13
`python3-numpy` / `python3-scipy` / `python3-pil`	1.26.4 / 1.11.4 / 10.2.0
`build-essential` / `cmake` / `git`	12.10ubuntu1 / 3.28.3 / 2.43.0
`tcpdump` / `tmux` / `gh`	4.99.4 / 3.4 / 2.45.0
`weasyprint` / `python3-markdown` (solo para generar este documento)	61.1 / 3.5.2

Anexo C. Archivos del repositorio

Repositorio: `github.com/EzeCombina/LiDAR-RobotHumanoide`.

Carpeta o archivo	Contenido
`README.md`	Presentación del proyecto, tabla de pasos y uso rápido
`docs/pasos/`	Un documento por paso (00 a 08), con comandos, salidas y problemas frecuentes
`docs/bitacora.md`	Registro de cada sesión de trabajo: qué se hizo, qué falló y cómo se resolvió
`docs/versiones.md`	Versiones exactas de todo lo instalado o descargado
`docs/grabaciones.md`, **`docs/mapas.md`**	Registro de grabaciones y mapas, y dónde están guardados
`docs/mediciones/`	Mediciones del sensor (vibración, estado interno)
`docs/referencias/`	Manuales oficiales del L2 y de Unilidar 2
`docs/pdf/`	Fuente de este documento (`fuente/`) y el PDF (`salida/`)
`config/unitree_lidar_ros2/l2_driver.launch.py`	Launch del driver con `modo:=ethernet|usb`
`config/rviz/`	Configuraciones de RViz para ver mapas 3D (`ver_mapa.rviz`) y 2D (`ver_mapa_2d.rviz`)
`scripts/sistema/`	`relevar_sistema.sh`: relevamiento del software instalado
`scripts/red/`	`configurar_red_lidar.sh`, `verificar_udp_lidar.py`
`scripts/sensor/`	Herramientas en C++ (`estado_lidar`, `cambiar_modo_lidar`) y `medir_vibracion_imu.py`
`scripts/grabacion/`	`grabar_lidar.sh`, `vigilar_datos.py`, `revisar_grabacion.py`
`scripts/mapas/`	`procesar_grabacion.sh`, `registrar_trayectoria.py`, `ver_mapa.sh`, `publicar_trayectoria.py`, `analizar_mapa` `verificar_mapa_2d.py`
`scripts/documentacion/generar_pdf.py`	Genera este documento
`bags/`	Grabaciones (fuera de git)
`mapas/<fecha>_<nombre>/`	Un mapa por carpeta: `mapa.pcd` y `mapa_limpio.pcd` (fuera de git), resúmenes, trayectorias y mapa 2D

Fuera del repositorio, en `~/lidar_ws/`: el SDK de Unitree (`unilidar_sdk2`, con el driver compilado) y Point-LIO (`point_lio_ws`).

C.1 Regenerar este documento

```
./scripts/documentacion/generar_pdf.py
```

Lee `docs/pdf/fuente/documento.yaml` (título, versión, fecha), los capítulos de `docs/pdf/fuente/capitulos/` y los estilos de `docs/pdf/fuente/estilos.css`, y genera el PDF en `docs/pdf/salida/`. Requiere `weasyprint` y `python3-markdown`.

Anexo D. Referencias

D.1 Documentación del fabricante

- Unitree Robotics. *Unitree 4D LiDAR L2 — User Manual*, v1.1 (2024).
- Unitree Robotics. *Unilidar 2 — User Manual*, v1.0 (2024).
- Página de descargas del L2: `https://www.unitree.com/download/L2`.

Copias en el repositorio: `docs/referencias/`.

D.2 Software

- SDK y driver ROS 2 del L2: `https://github.com/unitreerobotics/unilidar_sdk2`.
- Point-LIO para ROS 2 (versión usada): `https://github.com/dfloreaa/point_lio_ros2`.
- Point-LIO original (ROS 1): `https://github.com/hku-mars/Point-LIO`.
- Instalación de ROS 2 Jazzy en Ubuntu: `https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html`.
- Nav2 (navegación de ROS 2, servidor de mapas): `https://docs.nav2.org/`.
- WeasyPrint (generación de este PDF): `https://weasyprint.org/`.

D.3 Publicaciones

- D. He, W. Xu, N. Chen, F. Kong, C. Yuan y F. Zhang. *Point-LIO: Robust High-Bandwidth Light Detection and Ranging Inertial Odometry*. Advanced Intelligent Systems, 2023.